

School 21: ADM Patrol

Сценарий презентации — 10 слайдов, ~10 минут

1 Заголовок + демо (1 мин)

На слайде: название игры, QR-код на живую ссылку, скриншот геймплея

«Привет всем. Я хочу показать проект, который я сделала за последние несколько дней — это браузерная игра про нашу жизнь в кампусе. Называется School 21: ADM Patrol. Суть простая: ты студент, за тобой ходит ADM, и тебе надо убирать нарушения раньше чем он их заметит. Открывается прямо в телефоне, никаких установок. Можете отсканировать QR прямо сейчас и попробовать пока я рассказываю.»

2 Идея (30 сек)

На слайде: «что если сделать игру про то, что мы видим каждый день?»

«Идея появилась когда я в очередной раз убирала бутылку со стола перед приходом ADM. Я подумала — это же буквально механика игры. Есть угроза, есть таймер, есть действие. Взяла и сделала.»

3 Инструменты (1 мин)

На слайде: три иконки — Cursor, Claude, GitHub Pages

«Инструментов я использовала три. Cursor — это IDE с встроенным AI, я писала код прямо там, AI помогал генерировать куски логики и сразу объяснял что происходит. Claude — AI-ассистент, я с ним обсуждала архитектуру, он помогал когда что-то шло не так. И GitHub Pages — бесплатный хостинг от GitHub, деплой занял буквально две команды в терминале. Почему именно эти? Потому что всё это бесплатно, работает без сервера, и игру можно открыть на телефоне по ссылке.»

4 Технологический выбор (1 мин)

На слайде: один файл `index.html` vs фреймворки (React, Phaser — зачёркнуты)

«Первое решение которое я приняла — никаких библиотек. Только один HTML-файл. Почему? Потому что мне нужно было чтобы игра открывалась на любом телефоне без загрузки пакетов, без Node.js, без сервера. Скинула файл — открылось. Phaser.js — популярный игровой фреймворк — я рассматривала, но он весит 1 МБ и добавляет зависимость. Ванильный JS на Canvas делает то же самое, только я понимаю каждую строчку.»

5 Архитектура (1.5 мин)

На слайде: схема: Game Loop → Update → Draw → Input

«Основа любой игры — это игровой цикл. Каждые 16 миллисекунд браузер вызывает мою функцию. Она делает три вещи: обновляет состояние мира, проверяет столкновения, рисует на экране. Я разделила всё на четыре слоя. Input — обрабатывает тапы на мобиле и клики на десктопе. Update — двигаем ADM, спавним нарушения, считаем таймеры. Draw — рисуем всё на Canvas. И State — конечный автомат: меню, игра, терминал, конец игры. Каждое состояние независимо, переходы явные. Это стандартная архитектура для игр, её используют даже большие движки.»

6 Самое сложное: ADM (1.5 мин)

На слайде: картинка конуса обзора с формулой `Math.atan2`

«Самая интересная часть технически — это конус обзора ADM. Мне нужно было чтобы охранник видел только то, что находится перед ним в определённом угле. Решается через тригонометрию: функция `atan2` возвращает угол от охранника до любой точки. Сравниваю с текущим направлением взгляда — если разница меньше половины угла конуса, значит видит. Плюс проверка расстояния. Второй нетривиальный момент — я добавила `grace period`: нарушение появляется, но первые 1.5 секунды ADM его не может засчитать. Без этого игра была нечестная — объект спавнился и сразу пропадал, игрок ничего не успевал сделать.»

7 Мобильная адаптация (1 мин)

На слайде: два скриншота — портрет и ландшафт

«Изначально я делала под десктоп, и когда открыла на телефоне — всё было маленькое и не кликалось. Пришлось переписать под мобиль. Главное что я поменяла: Canvas работает в логических координатах 440 на 700, а под реальный экран масштабируется через CSS. Вся игровая логика написана один раз, а выглядит правильно и на маленьком айфоне и на большом андроиде. Тач-события

отдельные от мышинных — там есть нюанс с параметром `passive: false`, без него Safari добавляет 300 миллисекунд задержки на каждый тап.»

8 Геймдизайн (1 мин)

На слайде: таблица 5 уровней с параметрами

«Игра сбалансирована через конфиги уровней. Пять уровней, каждый по 60 секунд. Параметры: скорость ADM, интервал спауна нарушений, максимум одновременных нарушений. На первом уровне нарушение появляется раз в 2–4 секунды, ADM медленный. На пятом — три ADM, нарушения каждые 0.7 секунды. Комбо-система: три убранных подряд без ошибок дают множитель $\times 2$, шесть — $\times 3$, десять — $\times 5$. Это стандартный паттерн мобильных игр — мотивирует играть аккуратно.»

9 Деплой (1 мин)

На слайде: три шага с командами git

«Деплой занял три минуты. GitHub Pages — бесплатный хостинг для статичных файлов прямо из репозитория. Я создала ветку `gh-pages`, запустила туда файл, включила Pages в настройках репозитория. Всё. Игра доступна по адресу `username.github.io/daily_cod`. Никакого сервера, никакой оплаты, никакого CI. Идеально для прототипов — сделал, отправил ссылку, люди играют.»

10 Выводы (30 сек)

На слайде: три буллита: чему научилась / что работает / что бы добавила

«Что я из этого вынесла: браузер — это полноценная игровая платформа, не нужен Unity. AI-инструменты ускоряют работу не потому что пишут код за тебя, а потому что объясняют почему что-то не работает и предлагают варианты. И самое важное — начать с маленького рабочего прототипа, потом добавлять. Если хочется развить — добавила бы мультиплеер через WebSocket и звуки. Спасибо, попробуйте по QR.»

Тайминг

Слайд	Тема	Время
1	Демо	1:00

2	Идея	0:30
3	Инструменты	1:00
4	Технологический выбор	1:00
5	Архитектура	1:30
6	ADM и сложные моменты	1:30
7	Мобилка	1:00
8	Геймдизайн	1:00
9	Деплой	1:00
10	Выводы	0:30
Итого		10:00